

Modellauflösung

Katalog, Kennungen und die Stellen, an denen sich das Modell ändern lässt

Zweck

Der Modellkatalog in `rag_project.core.models` ist reine Daten plus dünne Zugriffshelfer. Er kennt weder HTTP-Clients noch Zugangsschlüssel — diese Trennung hält die Provider-Schicht frei von Modellwissen und lässt den Katalog unabhängig erweitern. Der Pfad ist Teil der Änderung: Das Modul liegt heute unter `core/models.py`, nicht mehr direkt im Paketwurzelvezeichnis.

Kennungen

Ein Modell hat zwei Kennungen. Die einfache ID ist der Name beim Anbieter, gepflegt ohne Präfix, etwa `ibm-granite/granite-4.2-8b`. Die qualifizierte Kennung ergänzt den Provider: `openrouter:z-ai/glm-5.3`. Nach außen wird immer die qualifizierte Kennung gezeigt, weil nur sie eindeutig ist.

Warum ohne Präfix pflegen

- Die ID wird im Katalog bewusst ohne Provider-Präfix geführt, weil der Provider aus dem Präfix des Anwendercodes abgeleitet wird. Ein Eintrag gehört genau einem Anbieter; das Präfix kommt erst beim Auflösen dazu.

Modelltypen

Jedes Modell trägt eine grobe Einordnung. Sie steuert Erwartungen an Kosten, Latenz und Eignung — und sie entscheidet mit, welche Parameter gesetzt werden.

Typ	Bedeutung
flagship	stärkste Qualität, höchste Kosten
balanced	Standardwahl für RAG
fast	klein und günstig, für Massenoperationen
reasoning	explizites Nachdenken, langsam aber gründlich
open_weight	offene Gewichte

Was ein Eintrag trägt

Feld	Bedeutung
id	Modell-ID beim Anbieter, ohne Provider-Präfix
provider	zugehöriger Anbieter
label	sprechender Name für Ausgaben und Logs
kind	Einordnung nach Modelltyp
context_window	Token-Fenster laut Anbieter, soweit bekannt
supports_temperature	ob der Parameter temperature erlaubt ist
default	ob es der Standard des Providers ist

Das Feld `supports_temperature` ist praktisch bedeutsam: Manche Reasoning-Modelle lehnen den Parameter `temperature` ab. Der Katalog sagt deshalb, ob er überhaupt gesetzt werden darf, statt dass ein Aufruf daran scheitert.

Wo sich das Modell ändern lässt

Das Modell lässt sich an vier Stellen bestimmen. Sie haben eine feste Rangfolge, und alle vier münden in dieselbe Funktion: `resolve_target()` in `llm/providers.py`. Wer nicht weiß, welche Stelle gerade zieht, sieht in der Regel ein Modell, das er nicht gewählt hat.

Die Rangfolge

Ebene	Wer ändert	Beständigkeit
Argument auf der Kommandozeile	Anwender	nur für diesen einen Lauf
Aufrufparameter im Python-Code	Anwender bzw. eigener Code	nur für diesen Aufruf bzw. dieses Objekt
Umgebungsvariable des Prozesses	Betreiber	für den laufenden Prozess
<code>.env</code> im Projektordner	Anwender	dauerhaft, bis die Datei geändert wird

Wer gewinnt

- Das Argument schlägt alles. Danach folgt CHAT_MODEL aus den Einstellungen, und zwar aus der Umgebungsvariable vor der .env. Ist beides nicht gesetzt, gilt der Standard des Providers aus dem Katalog.

1. Kommandozeile — der Schalter --model

Der einfachste Weg: Man hängt --model an den Aufruf. Der Wert gilt nur für diesen einen Lauf und überschreibt sowohl CHAT_MODEL als auch den Katalogstandard.

Befehl	Wirkung
uv run rag-project --check --model openrouter:z-ai/glm-5.3	Ziel für einen Lauf prüfen, ohne etwas auszusenden
uv run rag-project --model openai:gpt-4o-mini	Statusbericht mit erzwungenem Modell
uv run rag --ask "FRAGE" --model openrouter:z-ai/glm-5.3	eine Frage mit bestimmtem Modell beantworten
uv run rag --retrieve "FRAGE" --model openrouter:ibm-granite/granite-4.2-8b	Treffer ohne Modellaufruf
uv run chatbot --model openrouter:z-ai/glm-5.3	Chatbot mit bestimmtem Modell starten

Ein Präfix ist nicht zwingend. Ohne Präfix löst die Anwendung zuerst gegen den aktiven Provider auf; findet sich dort nichts, sucht der Katalog über alle Provider. Mit Präfix entscheidet das Präfix — und damit womöglich auch der Anbieter.

Befehl	Wirkung
uv run rag-project --check --model z-ai/glm-5.3	ohne Präfix, gegen CHAT_PROVIDER aufgelöst
uv run rag-project --check --model openai:gpt-4o-mini	Präfix erzwingt den Anbieter openai

Präfix wechselt den Anbieter

- Ein Modell eines anderen Anbieters braucht dessen Schlüssel. Der Aufruf mit `openai:gpt-4o-mini` scheitert mit dem Hinweis auf `OPENAI_API_KEY`, wenn nur der OpenRouter-Schlüssel gesetzt ist.

Welche Kennungen es gibt, zeigt der Katalog. Der Filter ist optional und nimmt `openai` oder `openrouter` an.

Befehl	Wirkung
<code>uv run rag-project --models</code>	gesamten Katalog mit Kennung, Name und Typ
<code>uv run rag-project --models openrouter</code>	nur die Modelle dieses Anbieters
<code>uv run rag-project --models openai</code>	nur die Modelle dieses Anbieters

Auch der Chatbot kennt einen eigenen Schalter. Er liest `--model` aus seinen Argumenten und setzt damit das Modell des Bots.

Befehl	Wirkung
<code>uv run chatbot</code>	Modell aus der <code>.env</code> bzw. der Katalogstandard
<code>uv run chatbot --model openrouter:z-ai/glm-5.3</code>	Modell für diese Sitzung
<code>uv run chatbot --no-stream</code>	Ausgabemodus vollständig statt stückweise

In der laufenden Sitzung zeigt `/model` das aktive Modell samt Provider, ohne etwas zu ändern. Zum Wechseln muss der Chatbot neu gestartet werden.

2. Python-Code — Argument und Klassenfeld

Wer die Bibliotheken direkt nutzt, übergibt das Modell als Argument. Es gilt nur für diesen Aufruf — oder, bei einer Klasse, für die Lebensdauer des Objekts.

Aufruf	Wirkung
<code>build_llm("openrouter:z-ai/glm-5.3")</code>	Client für dieses Modell bauen
<code>build_llm("openai:gpt-4o-mini", streaming=True)</code>	Client mit Streaming für ein anderes Modell
<code>build_llm(None)</code>	kein Modell erzwingen — <code>CHAT_MODEL</code> bzw. Katalogstandard greift

Aufruf	Wirkung
<code>resolve_target("openrouter:z-ai/glm-5.3")</code>	nur auflösen, nichts senden
<code>resolve_target(None)</code>	das aktive Ziel auflösen, wie <code>--check</code> es tut
<code>ChatBot(model="openrouter:z-ai/glm-5.3")</code>	Chatbot mit festem Modell für seine Lebensdauer

Der Unterschied zwischen den beiden letzten Zeilen ist die Beständigkeit. Ein Argument an `build_llm()` gilt für den einen Client; ein Feld am `ChatBot` gilt, bis das Objekt verworfen wird — und übersteht auch einen Reset des Verlaufs.

Ein Sonderfall ist der `ChatBot`: Er baut den Client beim Erzeugen und beim System-Reset neu. Das Modell bleibt dabei erhalten, weil es als Feld gespeichert ist. Wer das Modell während einer Sitzung ändern will, setzt das Feld und ruft `reset()` — oder erzeugt den Bot neu.

3. Umgebungsvariable des Prozesses

`CHAT_MODEL` lässt sich für einen einzelnen Aufruf in der Umgebung setzen. Der Wert gilt nur für diesen Prozess und überschreibt die `.env`, weil das Laden der `.env` keine bereits gesetzte Prozessvariable überschreibt.

Befehl	Wirkung
<code>\$env:CHAT_MODEL="openrouter:z-ai/glm-5.3"; uv run rag-project --check</code>	PowerShell: für diesen Aufruf setzen
<code>CHAT_MODEL="openrouter:z-ai/glm-5.3" uv run rag-project --check</code>	Linux und macOS: vorangestellt
<code>export CHAT_MODEL="openrouter:z-ai/glm-5.3"</code>	Linux und macOS: für die Sitzung setzen
<code>\$env:CHAT_MODEL=""</code>	PowerShell: Wert leeren, Standard greift wieder

Ein leerer Wert wirkt wie nicht gesetzt. Die Anwendung normalisiert den leeren String zu `None`, bevor sie auflöst — die Zeile darf also gefahrlos leer bleiben.

4. `.env` — dauerhaft für das Projekt

Der dauerhafte Weg: `CHAT_MODEL` in der `.env` in der Projektwurzel setzen, dort wo auch `pyproject.toml` liegt. Danach gilt das Modell für jeden Lauf, bis die Datei wieder geändert wird.

- `CHAT_MODEL=` — leer lassen: der Standard des gewählten Providers gilt
- `CHAT_MODEL=openrouter:z-ai/glm-5.3` — OpenRouter mit Präfix
- `CHAT_MODEL=openai:gpt-4o-mini` — OpenAI mit Präfix
- `CHAT_MODEL=z-ai/glm-5.3` — ohne Präfix, gegen `CHAT_PROVIDER` aufgelöst

Zusammen mit `CHAT_PROVIDER` ergibt sich der vollständige Zustand. Zeigt `CHAT_PROVIDER` auf openrouter und `CHAT_MODEL` auf ein OpenAI-Modell, meldet die Auflösung den Widerspruch, statt still das eine oder andere zu wählen.

Die Einstellungen werden einmal gelesen

- `get_settings()` erzeugt die Einstellungen genau einmal pro Prozess. Eine Änderung an der `.env` wirkt deshalb nicht in einem laufenden Programm — sie wirkt beim nächsten Start.

5. Katalog — ein Modell dauerhaft ergänzen

Soll ein Modell grundsätzlich zur Wahl stehen, gehört es in den Katalog. Das ist die einzige Stelle, an der ein Modell für alle verfügbar wird — und die einzige, die der Anwender nicht selbst ändert, sondern der Entwickler.

Ein neues Modell braucht genau einen Eintrag in der Rohliste in `core/models.py`: ohne Provider-Präfix, aber mit Anbieter, sprechendem Namen, Typ, Standardflag und Temperaturfreigabe. Das Registry prüft beim Aufbau auf doppelte IDs — ein zweiter Eintrag mit derselben Kennung wird abgelehnt, statt still den ersten zu überschreiben.

Die Auflösungsreihenfolge im Detail

Drei Quellen können das Modell bestimmen, und `resolve_target()` wendet sie in fester Reihenfolge an.

- Das Argument `model`, etwa über `--model` auf der Kommandozeile.
- `CHAT_MODEL` aus den Settings, sofern der Wert nicht leer ist.
- Der Standard des aktiven Providers aus dem Katalog.

Ein Anbieterpräfix hat Vorrang vor `CHAT_PROVIDER`. Wer `openai:gpt-5` schreibt, will OpenAI — auch wenn in der `.env` openrouter steht. Ohne Präfix löst der Katalog zuerst gegen den aktiven Provider auf; findet sich dort nichts, sucht er über alle Provider.

Mehrdeutigkeit und Mismatch

Zwei Fälle werden ausdrücklich abgewiesen, statt sie zu erraten. Erstens: Eine Kennung ohne Präfix, die bei mehreren Providern existiert, ist nicht auflösbar — der Katalog wirft `UnknownModelError`. Zweitens: Passt ein aufgelöstes Modell nicht zum aktiven Provider, meldet

die Provider-Schicht das mit einer Meldung, die beide Namen nennt und den Ausweg zeigt: CHAT_PROVIDER passend setzen oder ein Modell des aktiven Providers wählen.

Ergänzend gibt es eine unscharfe Wiederherstellung: Bei einer unbekannten Kennung liefert der Fehler Vorschläge, darunter den nächstliegenden Katalogeintrag. So wird aus einem Tippfehler eine brauchbare Meldung.

Befehl	Ergebnis
uv run rag-project --check --model openrouter:z-ai/glm-5.99	Fehler mit Vorschlägen
uv run rag-project --check --model gpt-5.6-luna	mehrdeutig ohne Präfix — Fehler statt Vermutung
uv run rag-project --check --model openai:gpt-4o-mini	Widerspruch zu CHAT_PROVIDER wird gemeldet

Kurz: wo ändere ich was?

Ich will ...	Ebene	Beispiel
einmalig ein anderes Modell	Kommandozeile	uv run rag --ask "FRAGE" --model openrouter:z-ai/glm-5.3
den Katalog ansehen	Kommandozeile	uv run rag-project --models
in einem eigenen Skript steuern	Python-Aufruf	build_llm("openrouter:z-ai/glm-5.3")
für eine Objektlaufzeit festlegen	Klassenfeld	ChatBot(model="openrouter:z-ai/glm-5.3")
nur für einen Prozess setzen	Umgebungsvariable	\$env:CHAT_MODEL="..."; uv run rag-project --check
dauerhaft für mich einstellen	.env	CHAT_MODEL=openrouter:z-ai/glm-5.3
ein Modell grundsätzlich verfügbar machen	Katalog	Eintrag in core/models.py ergänzen

Der Katalog in der Ausgabe

Die Projekt-CLI zeigt den Katalog mit --models, optional auf einen Provider gefiltert. Jede Zeile nennt die qualifizierte Kennung, den sprechenden Namen, den Typ und ob es sich um den

Standard handelt. Ein unbekannter Provider im Filter wird mit der Liste der gültigen Werte abgewiesen.

- `uv run rag-project --models` — der gesamte Katalog
- `uv run rag-project --models openai` — nur ein Anbieter
- `uv run rag-project --check` — löst das aktive Ziel auf und zeigt die Parameter

Testbezug

`tests/test_config_and_models.py` prüft unter anderem, dass jeder Provider genau einen Standard hat, dass die Kennung samt Präfix verlustfrei hin und zurück geht, dass eine unbekannte Kennung Vorschläge liefert, dass das Präfix die Einstellung schlägt und dass ein Modell ohne Temperatur-Unterstützung den Parameter nicht bekommt. Die Tests laufen ohne Netz.